


# Ciencia de la Computación

## Base de Datos II

Docente  Regina Paola  Ticona Herrera

Propuesta inicial del proyecto final

Entregado el 02/09/2026

Camila Mansilla
Nayalem Arune
Marccelo Tito
Rodrigo Fuentes
Pedro Flores

Semestre V

2026-2

"Los alumnos declaran haber realizado el presente trabajo de acuerdo a las normas de la Universidad Católica San Pablo"

_____ _____ _____ _____ _____

# Índice Skip List para PostgreSQL

1. **Estructura elegida y motivación.**

   Se seleccionó la estructura de datos Skip List como mecanismo alternativo de indexación integrado con PostgreSQL. Es una lista ordenada con varios niveles: cada nivel permite saltar sobre varios elementos, reduciendo el número de comparaciones necesarias durante una búsqueda, y a diferencia del B-tree no necesita reorganizarse para mantenerse equilibrada.
   La elegimos porque mantiene las claves ordenadas, lo que permite búsquedas por valor exacto y por rango, y porque es más sencilla de implementar y explicar que un árbol balanceado. Además se adapta bien al sistema de gestión de nuestro proyecto de BD1, que administra los recursos mediante identificadores numéricos.

2. **Referencias fundamentales**

   La principal referencia científica del proyecto es el artículo *Skip Lists: A Probabilistic Alternative to Balanced Trees*, publicado por William Pugh en 1990 (Communications of the ACM). Este trabajo introduce las Skip Lists, su organización multinivel y la forma en que se asignan los niveles de manera aleatoria. También usaremos la documentación oficial de PostgreSQL 18 sobre cómo crear nuevos tipos de índices mediante extensiones.

3. **Representación de la estructura y operaciones a implementar.**

   La Skip List se representará mediante nodos enlazados organizados en múltiples niveles. Cada nodo almacenará la clave entera (por ejemplo, recurso_id), la ubicación de la fila correspondiente en la tabla y los enlaces hacia los siguientes nodos en cada nivel. Contará con un nodo inicial (HEAD) y un nivel máximo definido.
   Operaciones obligatorias:
   - Construcción: creación de la Skip List a partir de los datos ya existentes en la tabla.
   - Búsqueda: localizar una clave existente o verificar que no se encuentra en la estructura.
   - Inserción: agregar una nueva clave manteniendo el orden y la estructura de niveles.

   Operaciones adicionales, si el tiempo lo permite:
   - Búsqueda por rango: recuperar las claves comprendidas entre dos valores.
   - Eliminación: retirar una clave existente (se deja como ampliación).

   En promedio, la búsqueda y la inserción tienen un costo logarítmico, O(log n), similar al de un B-tree.

4. **Tipo de clave y consultas soportadas**

   Se utilizarán claves enteras (INTEGER), ya que el proyecto de BD1 trabaja principalmente con identificadores numéricos. La estructura permitirá evaluar consultas de:
   - Igualdad: clave = valor.
   - Rango: clave entre valor1 y valor2.

   Las pruebas se harán con los conjuntos de datos de enteros indicados en las pautas (secuenciales, aleatorios y con acceso sesgado) en tamaños de 100 000, 500 000 y 1 000 000 registros, comparando siempre contra la búsqueda sin índice y contra el B-tree de PostgreSQL.

5. **Estrategia de integración con PostgreSQL.**

   La Skip List se integrará como un nuevo tipo de índice dentro de PostgreSQL, instalado como una extensión. De esta forma se podrá crear con una instrucción SQL normal (CREATE INDEX ... USING skiplist) y PostgreSQL la utilizará automáticamente al ejecutar consultas, sin depender de un programa externo.

La estructura y sus operaciones se implementarán en C++. Como PostgreSQL está escrito en C, se añadirá una pequeña capa en C que conecta ambos lados: PostgreSQL llama a esta capa al construir el índice, insertar datos o buscar, y ella invoca las funciones de la Skip List. Verificaremos que el índice se usa realmente mediante EXPLAIN ANALYZE.

6. **Identificación de qué componentes serán desarrollados por el equipo.**

   Se desarrollará desde cero la Skip List en C++, con su estructura de nodos y niveles y las operaciones de construcción, búsqueda e inserción. También se desarrollarán la capa de conexión con PostgreSQL, el entorno reproducible en Docker con su README, los generadores de datos y las pruebas y scripts de medición.

7. **Recursos externos, repositorios o bibliotecas que planean consultar o reutilizar.**

   Se consultará la documentación oficial de PostgreSQL, el artículo de Pugh y ejemplos de índices incluidos en el propio código de PostgreSQL como guía. Se utilizarán las bibliotecas estándar de C++, Git y Docker. Cualquier código externo o herramienta de IA que se utilice será indicado en el repositorio, diferenciando lo reutilizado de lo desarrollado por el equipo.

8. **Riesgos técnicos identificados y plan de trabajo del grupo.**

   Los principales riesgos identificados son:

   - Integración con PostgreSQL: comprender cómo PostgreSQL se comunica con un índice propio y conectar correctamente el código en C++ con PostgreSQL.
   - Manejo correcto de los niveles: un error en la asignación de niveles o en los enlaces podría producir búsquedas o inserciones incorrectas. Se mitigará con pruebas desde las primeras semanas.
   - Rendimiento: aunque el costo esperado es O(log n), depende del azar en los niveles; por eso cada medición se repetirá al menos 5 veces.

   **Plan de trabajo**

   - Semana 1: propuesta, diseño de la estructura y pseudocódigo.
   - Semana 2: construcción y búsqueda funcionando con pruebas; entorno Docker listo.
   - Semana 3: inserción y búsqueda por rango; inicio de la extensión.
   - Semana 4: índice utilizable desde PostgreSQL y corrección de errores.
   - Semana 5: demostración reproducible y primera comparación con B-tree.

   Algunos integrantes se enfocarán en la implementación de la Skip List y otros en la integración con PostgreSQL, el entorno, las pruebas y la documentación. Antes del parcial todos revisarán la solución completa para poder explicarla.